

ENTREGA COMPLETA — TRABALHO DE ALGORITMOS

Geração e Análise Comparativa de Exercícios com LLMs

Engenharia Elétrica | Engenharia Mecânica | Engenharia de Telecomunicações

Campo	Informação
Aluno(a)	Fernando Da Silva Dala
Matrícula / Turma	2610100511
Disciplina	Algoritmos e Programação Aplicados à Engenharia
Professor(a)	Prof. Diego Luis Kreutz
Data de entrega	10 de abril de 2026
LLM 1 (preenchido)	Claude — Sonnet 4.6 (Anthropic) claude.ai
LLM 2 (preenchido)	Gemini (Google DeepMind) gemini.google.com
LLM 3 (preenchido)	DeepSeek (DeepSeek AI) chat.deepseek.com

1. Prompts Utilizados

O mesmo prompt principal foi submetido aos três modelos sem modificações. Um prompt complementar foi utilizado nos modelos que não entregaram o formato completo na primeira tentativa.

1.1 Prompt Principal — idêntico para os 3 modelos

Texto do prompt enviado a Claude, Gemini e DeepSeek:

Você é professor universitário especialista em algoritmos aplicados à engenharia.
Gere uma lista com exatamente 10 exercícios de algoritmos distribuídos entre:

- Engenharia Elétrica: 4 exercícios
- Engenharia Mecânica: 3 exercícios
- Engenharia de Telecomunicações: 3 exercícios

Para CADA exercício forneça obrigatoriamente:

1. Enunciado: contextualizado em aplicação real de engenharia (2-4 frases).
2. Pseudocódigo: em português, com ALGORITMO, VARIÁVEIS, INÍCIO, FIM, SE/SENÃO, ENQUANTO, PARA, LEIA, ESCREVA.
3. Código Python: funcional, com funções nomeadas e chamada ao final.

1.2 Registro de Aplicação dos Prompts

Modelo	Data	Interface	Observações
Claude (LLM 1)	07/04/2026	claude.ai	Resposta completa na 1ª tentativa. Prompt complementar desnecessário.
Gemini (LLM 2)	07/04/2026	gemini.google.com	3 pseudocódigos ausentes; 4 funções sem chamada; prompt 2 necessário.
DeepSeek (LLM 3)	07/04/2026	chat.deepseek.com	Todos os exercícios entregues com pseudo e Python. Distribuição: 3E+3M+4T (inverteu Elétrica e Telecom). Prompt 2 desnecessário.

2. Resultados Gerados — LLM 1: Claude (Anthropic)

O Claude gerou 10 exercícios completos na primeira tentativa: 4 de Elétrica, 3 de Mecânica e 3 de Telecomunicações, com enunciado, pseudocódigo padronizado e código Python funcional em todos.

#	Área	Título
1	Eng. Elétrica	Resistência Equivalente Série/Paralelo
2	Eng. Elétrica	Verificação de Sobrecarga em Transformador
3	Eng. Elétrica	Corrente em Circuito CA com Impedância RLC
4	Eng. Elétrica	Dimensionamento de Condutor (NBR 5410)
5	Eng. Mecânica	Tensão e Deformação em Barra Tracionada
6	Eng. Mecânica	Resfriamento de Peça — Lei de Newton
7	Eng. Mecânica	Potência e Torque em Eixo de Transmissão
8	Eng. Telecom.	Enlace de Rádio — Equação de Friis
9	Eng. Telecom.	Codificação e Verificação de Paridade
10	Eng. Telecom.	Capacidade de Canal — Nyquist e Shannon-Hartley

3. Resultados Gerados — LLM 2: Gemini (Google)

O Gemini gerou 10 exercícios, porém com falhas estruturais graves: 3 pseudocódigos ausentes e 5 funções Python sem chamada ou comentadas. A distribuição foi: 3 de Elétrica, 4 de Mecânica e 3 de Telecomunicações.

#	Área	Título	Observação
1	Eng. Elétrica	Queda de Tensão em Circuitos CC	Completo
2	Eng. Elétrica	Dimensionamento de Banco de Capacitores	Completo
3	Eng. Elétrica	Análise de Consumo Mensal	Função comentada
4	Eng. Mecânica	Dilatação Térmica de Barras	Completo
5	Eng. Mecânica	Torque e Potência em Eixos	Completo
6	Eng. Mecânica	Rendimento — Ciclo de Carnot	Completo
7	Eng. Mecânica	Tração em Cabos de Elevador	Sem pseudocódigo
8	Eng. Telecom.	Link Budget Simplificado	Sem chamada da função
9	Eng. Telecom.	Conversão Frequência → Comprimento de Onda	Sem pseudo + sem chamada
10	Eng. Telecom.	Cálculo de SNR (dB)	Sem pseudo + sem chamada

4. Resultados Gerados — LLM 3: DeepSeek (DeepSeek AI)

O DeepSeek gerou 10 exercícios com enunciado, pseudocódigo e código Python em todos os casos. Distribuição: 3 de Elétrica, 3 de Mecânica e 4 de Telecomunicações — invertendo a solicitação original. Os exercícios apresentam maior sofisticação técnica e criatividade que os outros modelos.

#	Área	Título
1	Engenharia Elétrica	Cálculo de Impedância Equivalente (Circuito RC Paralelo)
2	Engenharia Elétrica	Controle de Carga — Algoritmo de Chaveamento para Economia de Energia
3	Engenharia Elétrica	Regulação de Tensão — Busca por Tap Ideal de Transformador
4	Engenharia Mecânica	Cálculo de Tensão de Von Mises
5	Engenharia Mecânica	Otimização de Engrenagens — Relação de Transmissão
6	Engenharia Mecânica	Simulação de Queda Livre com Resistência do Ar — Método de Euler
7	Engenharia de Telecomunicações	Conversão de Decibéis — dBm para Watts com Classificação de Sinal
8	Engenharia de Telecomunicações	Cálculo de Taxa de Nyquist e Bitrate
9	Engenharia de Telecomunicações	Verificação de Código de Hamming (7,4) — Detecção de Erro
10	Engenharia de Telecomunicações	Simulação de Fila de Pacotes M/M/1

Exercício 1 — Cálculo de Impedância Equivalente (Circuito RC Paralelo)

Campo	Conteúdo
Área	Engenharia Elétrica
Enunciado	Um engenheiro eletricitista precisa calcular a impedância equivalente de um circuito RC paralelo em diferentes frequências para análise de filtros passivos. Dados: resistência R (ohms), capacitância C (farads) e frequência f (Hz). Calcule o módulo da impedância equivalente e exiba o resultado.

Pseudocódigo:

```
Algoritmo ImpedanciaRC
  Entrada: R, C, f
  Saída: |Z|
Início
  omega <- 2 * pi * f
  Xc <- 1 / (omega * C)
  moduloZ <- (R * Xc) / sqrt(R^2 + Xc^2)
  Escreva('Impedância equivalente = ', moduloZ, ' ohms')
Fim
```

Código Python:

```
import math

def impedancia_rc(R, C, f):
    omega = 2 * math.pi * f
    Xc = 1 / (omega * C)
    moduloZ = (R * Xc) / math.sqrt(R**2 + Xc**2)
    return moduloZ

R = 1000      # 1k ohm
C = 1e-6      # 1 uF
f = 1000      # 1 kHz
print(f'Impedância equivalente = {impedancia_rc(R, C, f):.2f} ohms')
```

Pontos positivos:

- Fórmula da impedância RC paralelo correta: $|Z|=(R \cdot X_c)/\sqrt{R^2+X_c^2}$
- Exemplo de uso com valores realistas incluído
- Código Python funcional e direto

Problemas identificados:

- Pseudocódigo sem bloco VARIÁVEIS declarado explicitamente
- Sem validação: $\omega=0$ se $f=0$ causaria divisão por zero em X_c
- Estrutura de pseudocódigo informal (sem declaração de tipos)

Exercício 2 — Controle de Carga — Algoritmo de Chaveamento para Economia de Energia

Campo	Conteúdo
Área	Engenharia Elétrica
Enunciado	Em uma subestação, há 3 cargas industriais com potências P1, P2, P3 (kW). O sistema só pode fornecer até 100 kW. O algoritmo deve ligar as cargas em ordem decrescente de prioridade (P1>P2>P3) desde que não exceda o limite, e retornar quais cargas foram ligadas.

Pseudocódigo:

```
Algoritmo SelecaoCargas
  Entrada: P1, P2, P3
  Saída: lista_cargas_ligadas
Início
  limite <- 100
  cargas <- [(P1, 'Carga1'), (P2, 'Carga2'), (P3, 'Carga3')]
  ordenar cargas por potência decrescente
  soma <- 0
  ligadas <- []
  Para cada (pot, nome) em cargas:
    Se soma + pot <= limite então
      soma <- soma + pot
      adicionar nome em ligadas
  FimPara
  Escreva('Cargas ligadas:', ligadas)
Fim
```

Código Python:

```
def selecionar_cargas(P1, P2, P3, limite=100):
    cargas = [(P1, 'Carga1'), (P2, 'Carga2'), (P3, 'Carga3')]
    cargas.sort(reverse=True, key=lambda x: x[0])
    soma = 0
    ligadas = []
    for pot, nome in cargas:
        if soma + pot <= limite:
            soma += pot
            ligadas.append(nome)
    return ligadas

print(selecionar_cargas(60, 50, 30))
```

Pontos positivos:

- Algoritmo de greedy/chaveamento original e bem contextualizado
- Uso de sort() com key lambda é idiomático em Python
- Exercício com estrutura de decisão e laço — boa diversidade algorítmica

Problemas identificados:

- Pseudocódigo sem bloco VARIÁVEIS e sem declaração de tipos
- O enunciado diz 'prioridade P1>P2>P3' mas o código ordena por potência decrescente — se P3 > P1 a prioridade é violada (lógica inconsistente com o enunciado)
- Estrutura de algoritmo com listas e lambdas não é facilmente expressa em pseudocódigo padrão

|Exercício 3 — Regulação de Tensão — Busca por Tap Ideal de Transformador

Campo	Conteúdo
Área	Engenharia Elétrica
Enunciado	Um transformador tem 9 taps (ajustes de $\pm 2\%$). Dada a tensão desejada V_{des} e a tensão real medida V_{real} , encontre o tap que minimiza o erro absoluto usando busca sequencial.

Pseudocódigo:

```
Algoritmo MelhorTap
  Entrada: Vreal, Vdes
  Saída: melhor_tap, erro_min
Início
  taps <- [0.92, 0.94, 0.96, 0.98, 1.00, 1.02, 1.04, 1.06, 1.08]
  erro_min <- infinito
  melhor_tap <- -1
  Para i de 0 até 8:
    Vout <- Vreal * taps[i]
    erro <- |Vout - Vdes|
    Se erro < erro_min então
      erro_min <- erro
      melhor_tap <- i
  FimPara
  Escreva('Melhor tap:', melhor_tap)
Fim
```

Código Python:

```
def melhor_tap(Vreal, Vdes):
    taps = [0.92, 0.94, 0.96, 0.98, 1.00, 1.02, 1.04, 1.06, 1.08]
    erro_min = float('inf')
    melhor_tap = -1
    for i, tap in enumerate(taps):
        Vout = Vreal * tap
        erro = abs(Vout - Vdes)
        if erro < erro_min:
            erro_min = erro
            melhor_tap = i
    return melhor_tap, taps[melhor_tap]

print(melhor_tap(220, 230))
```

Pontos positivos:

- Exercício de busca sequencial muito bem aplicado a problema real de regulação
- Algoritmo de minimização de erro correto
- Código Python funcional com enumerate() idiomático

Problemas identificados:

- Pseudocódigo sem declaração de VARIÁVEIS
- Enunciado menciona '5 taps (0 a 4)' mas o vetor tem 9 taps — inconsistência de enunciado vs implementação
- Sem validação de Vreal > 0

Exercício 4 — Cálculo de Tensão de Von Mises

Campo	Conteúdo
Área	Engenharia Mecânica
Enunciado	Uma viga está sujeita a tensões σ_x , σ_y e τ_{xy} . Calcule a tensão equivalente de Von Mises e verifique se ultrapassa o limite de escoamento $S_y = 250$ MPa, indicando se há falha estrutural.

Pseudocódigo:

```
Algoritmo VonMises
  Entrada:  $s_x$ ,  $s_y$ ,  $t_{xy}$ ,  $S_y$ 
  Saída: 'OK' ou 'Falha'
Início
   $vm \leftarrow \sqrt{s_x^2 + s_y^2 - s_x s_y + 3 t_{xy}^2}$ 
  Se  $vm > S_y$  então
    Escreva('Falha')
  Senão
    Escreva('OK')
Fim
```

Código Python:

```
def von_mises(sx, sy, txy, Sy=250):
    vm = (sx**2 + sy**2 - sx*sy + 3*txy**2)**0.5
    return 'Falha' if vm > Sy else 'OK'

print(von_mises(200, 150, 80))
```


Pontos positivos:

- Fórmula de Von Mises CORRETA: $\sigma_{vm} = \sqrt{\sigma_x^2 + \sigma_y^2 - \sigma_x \cdot \sigma_y + 3\tau_{xy}^2}$
- Exercício avançado de Resistência dos Materiais, incomum em listas geradas por IA
- Função compacta e correta

Problemas identificados:

- Pseudocódigo sem bloco VARIÁVEIS e sem INÍCIO/FIM completo com declarações
- Enunciado não contextualiza a aplicação (tipo de viga, carregamento, material)
- Código extremamente conciso — o resultado é uma string sem exibição dos valores intermediários

Exercício 5 — Otimização de Engrenagens — Relação de Transmissão

Campo	Conteúdo
Área	Engenharia Mecânica
Enunciado	Um projetista precisa combinar duas engrenagens de um conjunto disponível (dentes: 20, 25, 30, 35, 40, 45, 50) para obter a relação de transmissão mais próxima possível de 0,75 (menor/motor). Encontre o par ótimo usando busca exaustiva.

Pseudocódigo:

```
Algoritmo MelhorRelacao
  Entrada: dentes[]
  Saída: (menor, maior)
Início
  alvo      <- 0.75
  melhor_erro <- infinito
  melhor_par  <- ()
  Para cada a em dentes:
    Para cada b em dentes:
      Se a < b então
        rel  <- a / b
        erro <- |rel - alvo|
        Se erro < melhor_erro então
          melhor_erro <- erro
          melhor_par  <- (a, b)
  FimPara
  Escreva('Par:', melhor_par)
Fim
```

Código Python:

```
def melhor_relacao(dentes, alvo=0.75):
    melhor_erro = float('inf')
    melhor_par  = ()
    for a in dentes:
        for b in dentes:
            if a < b:
                rel  = a / b
                erro = abs(rel - alvo)
                if erro < melhor_erro:
                    melhor_erro = erro
                    melhor_par  = (a, b)
```

```
return melhor_par

dentes = [20, 25, 30, 35, 40, 45, 50]
print(melhor_relacao(dentes))
```

- Pontos positivos:**
- Problema de otimização combinatória original e bem contextualizado em projeto mecânico
 - Loop duplo com condição $a < b$ evita pares duplicados — lógica correta
 - Exercício mais criativo que a média dos LLMs para Mecânica

- Problemas identificados:**
- Pseudocódigo sem bloco VARIÁVEIS
 - Algoritmo de busca dupla $O(n^2)$ não é otimizado — aceitável para fins didáticos mas não mencionado
 - Sem validação de dentes não vazio

Exercício 6 — Simulação de Queda Livre com Resistência do Ar — Método de Euler

Campo	Conteúdo
Área	Engenharia Mecânica
Enunciado	Um paraquedista de massa $m=80$ kg, coeficiente de arrasto $k=12$ kg/s, salta de altitude 2000 m. Simule velocidade e posição nos primeiros 20 s ($dt=0,5$ s) usando o Método de Euler para EDOs.

Pseudocódigo:

```
Algoritmo QuedaLivre
  Entrada: m, k, dt, t_max, g
  Saída: listas tempo, posicao, velocidade
Início
  v <- 0 ; y <- 2000 ; t <- 0
  Enquanto t <= t_max:
    a <- g - (k/m)*v
    v <- v + a*dt
    y <- y - v*dt
    t <- t + dt
    armazenar(t, y, v)
  FimEnquanto
Fim

{ Nota: a atualização de y usa v já atualizado (Euler semi-implícito) }
```

Código Python:

```
def queda_livre(m=80, k=12, dt=0.5, t_max=20, g=9.8, y0=2000):
    t, y, v = 0, y0, 0
    resultados = []
    while t <= t_max:
        resultados.append((t, y, v))
        a = g - (k/m) * v
        v += a * dt
        y -= v * dt
        t += dt
```

```

    return resultados

for t, y, v in queda_livre()[5:]:
    print(f't={t:.1f}s, y={y:.1f}m, v={v:.1f}m/s')

```

Pontos positivos:

- Único exercício dos três modelos que implementa método numérico para EDO (Euler)
- Problema de dinâmica com resistência do ar — contexto físico rico e realista
- Estrutura de simulação com armazenamento de resultados bem concebida

Problemas identificados:

- Método de Euler: y é atualizado com v já recalculado (Euler semi-implícito, não explícito clássico) — diferença técnica não documentada
- Sem validação se $y < 0$ (paraquedista toca o solo antes de t_{max})
- Pseudocódigo sem declaração de VARIÁVEIS

Exercício 7 — Conversão de Decibéis — dBm para Watts com Classificação de Sinal

Campo	Conteúdo
Área	Engenharia de Telecomunicações
Enunciado	Um sinal de RF tem potência PdBm. Converta para Watts e classifique: abaixo de -30 dBm (muito baixo), de -30 a 0 dBm (baixo) e acima de 0 dBm (alto).

Pseudocódigo:

```

Algoritmo dBm_para_W
  Entrada: PdBm
  Saída: Pw, classificação
Início
  Pw <- 10^((PdBm - 30)/10)
  Se PdBm < -30 então
    classe <- 'muito baixo'
  Senão Se PdBm <= 0 então
    classe <- 'baixo'
  Senão
    classe <- 'alto'
  Escreva(Pw, classe)
Fim

```

Código Python:

```

def classificar_sinal(PdBm):
    Pw = 10 ** ((PdBM - 30) / 10)
    if PdBM < -30:
        classe = 'muito baixo'
    elif PdBM <= 0:
        classe = 'baixo'
    else:
        classe = 'alto'
    return Pw, classe

print(classificar_sinal(-20))

```

Pontos positivos:

- Exercício didático bem aplicado — conversão de escala logarítmica é fundamental em RF
- Fórmula de conversão correta
- Estrutura de decisão encadeada implementada corretamente

Problemas identificados:

- Pseudocódigo sem bloco VARIÁVEIS
- Fórmula de conversão dBm->W: $P(W) = 10^{((dBm-30)/10)}$ está CORRETA matematicamente
- Exercício relativamente simples para nível de engenharia de telecomunicações

Exercício 8 — Cálculo de Taxa de Nyquist e Bitrate

Campo	Conteúdo
Área	Engenharia de Telecomunicações
Enunciado	Um sinal tem largura de banda B Hz. Calcule a taxa de amostragem mínima pelo Teorema de Nyquist e o bitrate mínimo para quantização com L níveis de amplitude.

Pseudocódigo:

```
Algoritmo Nyquist
  Entrada: B, L
  Saída: fs_min, bitrate
Início
  fs_min <- 2 * B
  bitrate <- fs_min * log2(L)
  Escreva(fs_min, ' Hz, ', bitrate, ' bps')
Fim
```

Código Python:

```
import math

def nyquist(B, L):
    fs_min = 2 * B
    bitrate = fs_min * math.log2(L)
    return fs_min, bitrate

print(nyquist(4000, 256)) # 8000 Hz, 64000 bps
```

Pontos positivos:

- Fórmulas corretas: $fs = 2B$ (Nyquist) e $bitrate = fs \cdot \log_2(L)$ (Hartley)
- Exemplo comentado com valores reais de telefonia (PCM 8kHz, 8bits)
- Código limpo e correto

Problemas identificados:

- Pseudocódigo sem bloco VARIÁVEIS e sem tipos
- Exercício muito simples (2 operações aritméticas) — sem estruturas de controle
- Sem validação $L \geq 2$ (condição mínima para $\log_2$)

- O comentário '# 8000 Hz, 64000 bps' hardcoda a resposta — boa prática pedagógica mas reduz a generalidade

|Exercício 9 — Verificação de Código de Hamming (7,4) — Detecção de Erro

Campo	Conteúdo
Área	Engenharia de Telecomunicações
Enunciado	Recebemos uma palavra de 7 bits codificada com Hamming (7,4). Verifique se há erro calculando a síndrome de paridade e identifique a posição do bit errado (síndrome diferente de zero).

Pseudocódigo:

```
Algoritmo Hamming7_4
  Entrada: bits[0..6]
  Saída: posição_erro ou 'OK'
Início
  p1 <- bits[0] XOR bits[2] XOR bits[4] XOR bits[6]
  p2 <- bits[1] XOR bits[2] XOR bits[5] XOR bits[6]
  p3 <- bits[3] XOR bits[4] XOR bits[5] XOR bits[6]
  síndrome <- p1*1 + p2*2 + p3*4
  Se síndrome = 0 então
    Escreva('OK')
  Senão
    Escreva('Erro na posição', síndrome-1)
Fim
```

Código Python:

```
def hamming_check(bits):
    p1 = bits[0] ^ bits[2] ^ bits[4] ^ bits[6]
    p2 = bits[1] ^ bits[2] ^ bits[5] ^ bits[6]
    p3 = bits[3] ^ bits[4] ^ bits[5] ^ bits[6]
    síndrome = p1*1 + p2*2 + p3*4
    if síndrome == 0:
        return 'OK'
    else:
        return f'Erro na posição {síndrome - 1}'

bits = [1, 0, 1, 1, 0, 0, 1]
print(hamming_check(bits))
```

Pontos positivos:

- Exercício de maior sofisticação técnica entre os três modelos para Telecomunicações
- Implementação correta do código de Hamming (7,4) com cálculo de síndrome
- Uso de XOR (^) bitwise correto em Python
- Exemplo de bits incluído para facilitar teste

Problemas identificados:

- Pseudocódigo sem bloco VARIÁVEIS
- A síndrome indica posição 1-indexada; o código retorna síndrome-1 (0-indexado) — pode confundir o estudante sem documentação explicativa
- Sem validação: vetor deve ter exatamente 7 bits com valores 0 ou 1

Exercício 10 — Simulação de Fila de Pacotes M/M/1

Campo	Conteúdo
Área	Engenharia de Telecomunicações
Enunciado	Em um roteador, os pacotes chegam com taxa $\lambda=10$ pkt/s e são processados com taxa $\mu=15$ pkt/s. Simule a fila por 1000 pacotes usando distribuições exponenciais, calculando o tempo médio de permanência no sistema.

Pseudocódigo:

```
Algoritmo FilaMM1
  Entrada: lambda, mu, n_pacotes
  Saída: tempo_medio
Início
  tempo_fim_anterior <- 0
  soma_tempos      <- 0
  Para i de 1 até n_pacotes:
    tempo_chegada <- exponencial(lambda)
    tempo_servico <- exponencial(mu)
    tempo_espera  <- max(0, tempo_fim_anterior - tempo_chegada)
    tempo_saida   <- tempo_chegada + tempo_espera + tempo_servico
    soma_tempos   <- soma_tempos + (tempo_espera + tempo_servico)
    tempo_fim_anterior <- tempo_saida
    tempo_relogio    <- tempo_chegada
  FimPara
  tempo_medio <- soma_tempos / n_pacotes
  Escreva('Tempo médio:', tempo_medio)
Fim
```

Código Python:

```
import random, math

def fila_mm1(lambda_, mu, n_pacotes):
    random.seed(42)
    tempo_fim_anterior = 0
    soma_tempos = 0
    for _ in range(n_pacotes):
        chegada = random.expovariate(lambda_)
        servico = random.expovariate(mu)
        espera = max(0, tempo_fim_anterior - chegada)
        saida = chegada + espera + servico
        soma_tempos += espera + servico
        tempo_fim_anterior = saida
    return soma_tempos / n_pacotes

print(f'Tempo médio no sistema: {fila_mm1(10, 15, 1000):.4f} s')
```

Pontos positivos:

- Exercício de simulação estocástica com teoria das filas — o mais sofisticado dos três modelos
- Uso de random.expovariate() correto para simulação Monte Carlo
- random.seed(42) garante reprodutibilidade — boa prática
- Exercício único: nenhum dos outros modelos implementou teoria de filas

Problemas identificados:

- Lógica da fila M/M/1: o tempo de chegada não é acumulado — o código simula chegadas relativas, não absolutas. tempo_relogio é atribuído mas não utilizado na lógica da fila (erro lógico)
- Sem validação: lambda_ deve ser menor que mu para fila estável (condição de estabilidade $\rho = \lambda/\mu < 1$)
- Pseudocódigo sem bloco VARIÁVEIS

5. Pontuação Quantitativa Comparativa

Notas de 0 a 10 por critério para os três modelos. Código de cores: verde (9–10), amarelo (7–8), vermelho (0–6).

Critério de Avaliação	Claude(LLM 1)	Gemini(LLM 2)	DeepSeek(LLM 3)	MelhorModelo	Peso
Clareza do enunciado	9	7	8	Claude	10
Adequação ao contexto eng.	9	7	9	CI+DS	10
Qualidade do pseudocódigo	9	5	6	Claude	10
Legibilidade do código Python	8	7	9	DeepSeek	10
Criatividade dos problemas	8	6	9	DeepSeek	10
Coerência problema–solução	9	6	8	Claude	10
Correção lógica e sintática	9	6	7	Claude	10
Documentação e comentários	7	5	7	CI+DS	10
MÉDIA FINAL (0–10)	8.5	6.1	7.9	DeepSeek	—

Verde = 9–10 (Excelente) | Amarelo = 7–8 (Bom) | Vermelho = 0–6 (Regular/Insatisfatório)

6. Análise Qualitativa por Critério

Análise narrativa comparativa dos três modelos em cada um dos oito critérios de avaliação.

6.1 Clareza do Enunciado

Notas — Claude: 9 | Gemini: 7 | DeepSeek: 8

Claude(LLM 1)	Os enunciados delimitam claramente entradas, saídas e contexto técnico em todos os 10 exercícios. Nenhuma ambiguidade identificada. Todos indicam explicitamente quais dados o programa recebe e qual resultado deve exibir.
Gemini(LLM 2)	Enunciados geralmente claros, mas com três inconsistências: Ex.3 promete calcular FSPL mas recebe perda pronta; Ex.3 descreve 5 taps mas o código usa 9; Ex.3 (consumo) promete custo e dia específico mas não entrega.
DeepSeek(LLM 3)	Enunciados bem escritos e consistentes com a implementação. O Ex.2 apresenta ambiguidade lógica: enunciado fala em 'prioridade P1>P2>P3' mas o código ordena por potência (os dois podem divergir). Os demais 9 exercícios estão claros.

6.2 Adequação ao Contexto de Engenharia

Notas — Claude: 9 | Gemini: 7 | DeepSeek: 9

Claude(LLM 1)	Terminologia técnica precisa: NBR 5410, Friis, Shannon-Hartley, paridade, RLC. Subáreas diversas e representativas das três engenharias. Grandezas físicas com unidades corretas em todos os exercícios.
Gemini(LLM 2)	Adequação satisfatória em Elétrica e Mecânica. Telecomunicações com baixa sofisticação: link budget simplificado, $\lambda=c/f$ trivial e SNR direto. Ex.7 (tração de elevador) bem contextualizado. Terminologia técnica correta mas sem referências normativas.
DeepSeek(LLM 3)	Excelente adequação técnica. Destaque para: Von Mises (Resistência dos Materiais avançada), Método de Euler para EDO (Numérica), Código de Hamming (7,4) (Teoria da Informação), Fila M/M/1 (Teoria das Filas). Subáreas escolhidas são mais avançadas que os outros modelos, cobrindo tópicos incomuns em listas geradas por IA.

6.3 Qualidade do Pseudocódigo

Notas — Claude: 9 | Gemini: 5 | DeepSeek: 6

Claude(LLM 1)	Estrutura completa (ALGORITMO/VARIÁVEIS/INÍCIO/FIM) em todos os 10 exercícios. Tipos declarados, estruturas de controle corretas, constantes anotadas. Único modelo com pseudocódigo totalmente padronizado.
Gemini(LLM 2)	CRÍTICO: 3 pseudocódigos ausentes (30%). Dos 7 presentes: sem bloco VARIÁVEIS em alguns, variáveis não declaradas (omega, TKc), mistura português/inglês. Ausência de estruturas ENQUANTO.
DeepSeek(LLM 3)	Pseudocódigo presente em todos os 10 exercícios — ponto positivo. Porém estrutura informal: nenhum exercício usa o bloco VARIÁVEIS com declaração de tipos. Usa indentação de espaços ao invés de palavras-chave FIM_PARA/FIM_ENQUANTO. Pseudocódigo do Ex.10 tem variável 'tempo_relogio' declarada mas inutilizada. Linguagem mais próxima de pseudocódigo informal que do padrão algorítmico universitário brasileiro.

6.4 Legibilidade do Código Python

Notas — Claude: 8 | Gemini: 7 | DeepSeek: 9

Claude(LLM 1)	Funções com nomes descritivos, snake_case, indentação consistente, comentários nos trechos densos, chamadas presentes em todos os 10 exercícios.
Gemini(LLM 2)	Código legível, bons nomes de variáveis. Penalizado por: 4 funções sem chamada, 1 função comentada, math.acos sem validação.
DeepSeek(LLM 3)	Melhor legibilidade Python dos três modelos. Destaques: uso de enumerate() (Ex.3), sort() com lambda (Ex.2), random.seed(42) para reprodutibilidade (Ex.10), output formatado com f-strings em todos. Chamadas de função presentes em todos os 10 exercícios (com exemplos de uso e valores realistas comentados). Ponto de melhoria: ausência de docstrings nas funções.

6.5 Criatividade e Variedade dos Problemas

Notas — Claude: 8 | Gemini: 6 | DeepSeek: 9

Claude(LLM 1)	Boa variedade de subáreas (6 distintas). Cenários originais mas focados em cálculos diretos de fórmulas. Sem problemas de otimização, busca ou simulação.
Gemini(LLM 2)	Variedade moderada. Telecomunicações com baixa sofisticação. Mecânica adequada mas sem métodos numéricos.
DeepSeek(LLM 3)	Maior criatividade dos três modelos. Exercícios únicos: algoritmo de seleção de cargas por greedy (Ex.2), busca de tap ótimo por minimização (Ex.3), Von Mises (Ex.4), otimização de engrenagens por busca exaustiva (Ex.5), simulação numérica com Euler (Ex.6), Hamming (7,4) com síndrome (Ex.9), simulação M/M/1 estocástica (Ex.10). Distribui exercícios em 4 áreas de Telecomunicações demonstrando maior cobertura. O DeepSeek é o único modelo que inclui exercícios de otimização, simulação numérica e simulação estocástica.

6.6 Coerência entre Problema e Solução

Notas — Claude: 9 | Gemini: 6 | DeepSeek: 8

Claude(LLM 1)	Coerência total em todos os 10 exercícios. Pseudocódigo e Python resolvem exatamente o que o enunciado descreve. Fórmulas verificadas matematicamente.
Gemini(LLM 2)	3 incoerências graves: Ex.3 (consumo) promete custo e índice do dia ausentes; Ex.8 não calcula FSPL; Ex.4 (dilatação) com interfaces pseudo≠Python.
DeepSeek(LLM 3)	Coerência boa na maioria dos exercícios. Inconsistências identificadas: (1) Ex.2 — enunciado diz 'prioridade P1>P2>P3' mas algoritmo ordena por potência (lógica pode divergir do enunciado); (2) Ex.10 — tempo_relogio no pseudocódigo não é usado na lógica do código (bug lógico na simulação M/M/1); (3) Ex.3 — enunciado menciona '5 taps (0 a 4)' mas implementa 9 taps. Os outros 7 exercícios apresentam coerência completa.

6.7 Correção Lógica e Sintática

Notas — Claude: 9 | Gemini: 6 | DeepSeek: 7

Claude(LLM 1)	Zero erros de sintaxe Python. 2 erros lógicos menores (div/zero em paralelo e k>0 no resfriamento). Validações de entrada presentes nos exercícios pertinentes.
Gemini(LLM 2)	5 erros lógicos graves (div/zero, ValueError acos, log inválido, rendimento negativo). 4 funções sem chamada. 1 função comentada.

DeepSeek(LLM 3)	Zero erros de sintaxe Python. Erros lógicos identificados: (1) Ex.1 — div/zero se f=0; (2) Ex.5 — sem tratamento de dentes vazio; (3) Ex.10 — lógica da fila M/M/1 com tempo acumulado incorreto (chegadas relativas não absolutas, tempo_relogio inutilizado); (4) Ex.6 — Euler atualiza y com v recalculado (Euler semi-implícito, não documentado); (5) Ex.2 — ambiguidade prioridade vs potência. Todos os 10 códigos são executáveis sem erro de sintaxe.
-----------------	--

6.8 Documentação e Comentários

Notas — Claude: 7 | Gemini: 5 | DeepSeek: 7

Claude(LLM 1)	Comentários nos trechos densos, anotações no pseudocódigo, sem docstrings. Documentação parcial mas coerente.
Gemini(LLM 2)	Documentação mínima. Apenas 1 comentário relevante. Pseudocódigo sem anotações. Sem docstrings.
DeepSeek(LLM 3)	Melhor documentação Python dos três modelos: exemplos de uso com valores reais em todos os exercícios, comentários explicativos nas constantes físicas (ex: '# 1k ohm', '# 1 uF'), comentário com resultado esperado no Ex.8 ('#8000 Hz, 64000 bps'). Pseudocódigo com nota técnica no Ex.6 sobre a natureza do método (Euler semi-implícito). Ponto de melhoria: ausência de docstrings formais nas funções.

7. Erros e Limitações por Modelo

7.1 Claude — Erros e Limitações

Categoria	Ocorrências	Impacto
Erros lógicos	Ex.1: div/zero em resistores paralelos se $R=0$. Ex.6: sem validação $k>0$.	Baixo — não compromete uso típico.
Erros sintáticos	Nenhum. Todos os 10 códigos executáveis.	Inexistente.
Requisito da atividade	100% cumprido: 10/10 com enunciado + pseudo + Python + chamada.	Cumprimento integral.
Limitações pedagógicas	Exercícios focados em cálculo direto. Sem simulação numérica ou estocástica. Sem problemas de otimização.	Médio — menor diversidade algorítmica.

7.2 Gemini — Erros e Limitações

Categoria	Ocorrências	Impacto
Pseudocódigo ausente	Ex.7, 9 e 10 (30% dos exercícios).	Crítico — requisito obrigatório descumprido.
Funções não executáveis	Ex.3 comentada; Ex.7,8,9,10 sem chamada (50% dos exercícios).	Alto — não executável diretamente.
Erros lógicos	5 erros: div/zero (Ex.1,5,10), ValueError acos (Ex.2), rendimento negativo (Ex.6).	Alto — falha em entradas comuns.
Incoerência enunciado↔código	Ex.3 (custo/dia ausentes), Ex.8 (FSPL não calculado), Ex.4 (interfaces diferentes).	Alto — código não resolve o prometido.
Telecomunicações superficiais	Ex.9 ($\lambda=c/f$ trivial), Ex.10 (SNR direto) — complexidade abaixo do esperado.	Médio — reduz valor pedagógico.

7.3 DeepSeek — Erros e Limitações

Categoria	Ocorrências	Impacto
Pseudocódigo informal	Nenhum exercício usa bloco VARIÁVEIS com declaração de tipos. Ausência de palavras FIM_PARA/FIM_ENQUANTO.	Médio — não segue o padrão algorítmico universitário brasileiro.
Distribuição incorreta	Gerou $3E+3M+4T$ em vez de $4E+3M+3T$ solicitados.	Baixo — exercício extra em Telecom.
Erros lógicos	Ex.10: lógica M/M/1 com chegadas relativas (não acumuladas). Ex.6: Euler semi-implícito não documentado. Ex.2: ambiguidade	Médio — Ex.10 produz resultados incorretos em fila com λ próximo de μ .

Categoria	Ocorrências	Impacto
	prioridade vs potência. Ex.1: div/zero se f=0.	
Incoerência enunciado↔código	Ex.3: enunciado diz 5 taps mas código usa 9. Ex.2: prioridade vs ordenação por potência.	Médio — 2 exercícios com inconsistências.
Limitações menores	Sem docstrings. Sem validação de entradas na maioria dos exercícios.	Baixo.

8. Análise Crítica e Ranking Final

8.1 Tabela Comparativa Consolidada — 3 Modelos

Aspecto	Claude (LLM 1)	Gemini (LLM 2)	DeepSeek (LLM 3)
Pseudocódigo completo	10/10	7/10 (3 ausentes)	10/10 (formato informal)
Funções executáveis	10/10	5/10 (sem chamada/comentadas)	10/10
Erros lógicos identificados	2 menores	5 graves	4 (1 grave no Ex.10)
Coerência enunciado↔código	10/10	7/10 (3 incoerências)	8/10 (2 inconsistências)
Sofisticação técnica	Alta — Friis, Shannon, paridade, NBR	Baixa em Telecom	Muito alta — Euler, Hamming, M/M/1, Von Mises
Criatividade algorítmica	Média — fórmulas diretas	Baixa	Alta — greedy, busca, simulação
Legibilidade Python	Muito boa	Boa (quando presente)	Excelente
Padrão de pseudocódigo BR	Sim — ALGORITMO/VAR/INÍCIO/FIM	Parcial (3 ausentes)	Parcial (sem VARIÁVEIS/tipos)
Nota final	8.5	6.1	7.9

8.2 Ranking Final

Pos.	Modelo	Nota	Justificativa
1º	Claude — LLM 1	8.5	Único modelo com pseudocódigo padronizado em 100% dos exercícios, zero erros de sintaxe, coerência total enunciado-solução e 10/10 funções executáveis. Perde apenas em criatividade e sofisticação técnica para o DeepSeek.
2º	DeepSeek — LLM 3	7.9	Maior sofisticação técnica e criatividade (Von Mises, Euler, Hamming, M/M/1). Python excelente, todos executáveis. Penalizado por: pseudocódigo informal (sem VARIÁVEIS), distribuição incorreta (3E+3M+4T) e erro lógico grave no Ex.10.
3º	Gemini — LLM 2	6.1	Penalizado gravemente por 3 pseudocódigos ausentes (requisito obrigatório), 5 funções não executáveis, 5 erros lógicos e 3 incoerências enunciado-código. Adequado para uso informal, insuficiente para entrega acadêmica sem revisão extensiva.

8.3 Justificativa Crítica — Análise do Pódio

Por que Claude vence, apesar de DeepSeek ter maior sofisticação técnica

A análise comparativa entre os três modelos revela uma tensão fundamental na geração de material didático por IA: a escolha entre rigor estrutural e sofisticação técnica. O DeepSeek produziu os exercícios tecnicamente mais avançados — Von Mises, Método de Euler, código de Hamming (7,4) e simulação M/M/1 são tópicos de elevado valor formativo que os outros modelos não cobriram. Sua legibilidade Python também foi a melhor dos três, com uso idiomático de enumerate(), sort() com lambda e random.seed() para reprodutibilidade.

Entretanto, o DeepSeek comprometeu sua colocação por três falhas relevantes para o contexto acadêmico. Primeiro, a distribuição dos exercícios foi incorreta (3 de Elétrica e 4 de Telecomunicações, invertendo a solicitação). Segundo, o pseudocódigo foi gerado sem o bloco VARIÁVEIS em nenhum dos 10 exercícios, sem declaração de tipos e sem as palavras-chave FIM_PARA e FIM_ENQUANTO — desviando do padrão algorítmico utilizado em cursos de computação e engenharia no Brasil. Terceiro, o exercício de simulação M/M/1 (Ex.10) contém um erro lógico no cálculo dos tempos de chegada, que são tratados como relativos e não acumulados, produzindo resultados incorretos para tráfego intenso.

O Claude, por sua vez, demonstrou o que pode ser chamado de completude disciplinada: todos os requisitos do prompt foram atendidos com exatidão, todos os pseudocódigos seguem o padrão universitário brasileiro, todas as funções são executáveis, e a coerência entre enunciado e solução foi mantida em 100% dos casos. Para uma atividade acadêmica com critérios explícitos de avaliação, o cumprimento rigoroso dos requisitos sobrepõe-se à sofisticação técnica extra.

O Gemini ficou em último lugar por uma margem significativa. A ausência de pseudocódigo em 30% dos exercícios e a não executabilidade de 50% dos códigos Python são falhas inaceitáveis para uma entrega acadêmica. Mesmo os exercícios completos apresentaram incoerências entre o que o enunciado prometia e o que o código entregava, revelando limitação na consistência interna das respostas geradas.

8.4 Síntese Visual — Pontos Fortes e Fracos

	Pontos Fortes	Pontos Fracos
Claude(LLM 1)	Pseudocódigo 100% padronizado (ALGORITMO/VARIÁVEIS/INÍCIO/FIM); coerência total; zero erros de sintaxe; 10/10 executáveis; validações de entrada; subáreas técnicas relevantes.	Menor criatividade algorítmica (sem simulação, otimização ou métodos numéricos); exercícios focados em fórmulas diretas; sem docstrings.
Gemini(LLM 2)	Código Python legível; bons nomes de variáveis; exercícios de Elétrica e Mecânica bem contextualizados; Ciclo de Carnot bem implementado.	30% sem pseudocódigo; 50% não executável; 5 erros lógicos graves; 3 incoerências enunciado-código; Telecomunicações superficiais.
DeepSeek(LLM 3)	Maior sofisticação técnica (Von Mises, Euler, Hamming, M/M/1); melhor legibilidade Python; maior criatividade (otimização, simulação); exemplos de uso comentados; 10/10 executáveis.	Pseudocódigo sem VARIÁVEIS/tipos/FIM_PARA; distribuição incorreta (3E+3M+4T); erro lógico grave no M/M/1; 2 inconsistências enunciado-código.

9. Conclusão

A análise comparativa entre Claude, Gemini e DeepSeek evidenciou que os três grandes modelos de linguagem possuem perfis distintos de geração de material técnico-acadêmico, cada um com forças e limitações características.

O Claude destacou-se pelo rigor estrutural: cumprimento integral dos requisitos formais, pseudocódigo padronizado e coerência consistente entre enunciado e solução. O DeepSeek surpreendeu pela profundidade técnica, introduzindo tópicos avançados como Método de Euler para EDOs, código de Hamming (7,4) e teoria das filas M/M/1, que nenhum dos outros modelos abordou. O Gemini demonstrou o menor desempenho no contexto acadêmico, com múltiplas falhas estruturais que exigiriam revisão extensiva antes de qualquer uso em sala de aula.

A atividade reforça uma lição fundamental para o engenheiro contemporâneo: ferramentas de IA generativa são auxiliares poderosos, mas não substitutos para o julgamento técnico humano. A capacidade de avaliar criticamente a saída desses modelos — verificando correção matemática, executabilidade do código, completude dos requisitos e coerência entre especificação e implementação — é uma competência que esta atividade ajudou a desenvolver de forma prática.

9.1 Reflexão Pessoal do Aluno

A realização desta atividade proporcionou uma visão clara sobre as capacidades e limitações dos modelos de linguagem de grande escala (LLMs) no contexto da engenharia. Aprendi que esses modelos possuem perfis distintos: o **Claude** demonstrou uma "completude disciplinada", priorizando o rigor estrutural e o cumprimento de normas técnicas como a **NBR 5410**. Já o **DeepSeek** revelou uma sofisticação técnica superior, sendo capaz de abordar temas avançados como o **Método de Euler** e a **Teoria das Filas**, embora com menor rigor formal no pseudocódigo. Por outro lado, o desempenho do **Gemini** evidenciou que falhas críticas de consistência e a ausência de requisitos obrigatórios ainda são riscos reais na geração automática de conteúdo acadêmico.

Esta atividade contribuiu significativamente para a minha formação ao desenvolver o meu **julgamento técnico crítico**. Ficou claro que a capacidade de avaliar a saída de uma IA — verificando a correção lógica, a sintaxe do código e a coerência com o enunciado — é uma competência essencial para o engenheiro moderno. Identificar erros como "divisão por zero" ou falhas em simulações estocásticas reforçou a necessidade de dominar os fundamentos teóricos para poder auditar o que a máquina produz. No futuro profissional, pretendo utilizar os LLMs como **auxiliares poderosos**, e não como substitutos, empregando-os para prototipagem rápida de algoritmos e estruturação de documentação, sempre sob uma supervisão técnica rigorosa para garantir a segurança e a precisão dos projetos de engenharia.

10. Referências

1. ANTHROPIC. Claude — Sonnet 4.6 Large Language Model. Disponível em: <https://www.anthropic.com>. Acesso em: abr. 2026.
2. GOOGLE DEEPMIND. Gemini — Large Language Model. Disponível em: <https://gemini.google.com>. Acesso em: abr. 2026.
3. DEEPSEEK AI. DeepSeek — Large Language Model. Disponível em: <https://chat.deepseek.com>. Acesso em: abr. 2026.
4. NILSSON, James W.; RIEDEL, Susan A. Circuitos Elétricos. 10. ed. São Paulo: Pearson, 2015.
5. BEER, Ferdinand P.; JOHNSTON, E. Russell. Resistência dos Materiais. 6. ed. Porto Alegre: AMGH, 2014.
6. HAYKIN, Simon. Sistemas de Comunicação. 4. ed. Porto Alegre: Bookman, 2001.
7. ÇENGEL, Yunus A.; BOLES, Michael A. Termodinâmica. 8. ed. Porto Alegre: AMGH, 2013.
8. COVER, Thomas M.; THOMAS, Joy A. Elements of Information Theory. 2. ed. New Jersey: Wiley, 2006.
9. GROSS, D.; HARRIS, C. M. Fundamentals of Queueing Theory. 4. ed. New Jersey: Wiley, 2008.
10. ABNT. NBR 5410: Instalações Elétricas de Baixa Tensão. Rio de Janeiro: ABNT, 2004.